МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ УНИВЕРСИТЕТ АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА № 41

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

Старший преподаватель		Т.Н. Григорьева
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Построение конечного автомата по блок-схеме алгоритма распознавания

по курсу: Теория автоматов и формальных языков

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. № 4417		М. П. Курбатов
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

Лабораторная работа № 3

Построение распознающего автомата с магазинной памятью

Цель работы

По результатам лабораторной работы № 2 для варианта 13 построить КС-грамматику, эквивалентный конечный распознаватель и МП-автомат; сравнить результаты распознавания и разобрать изменение стека.

1 Задание и связь с предыдущей работой

Исходный алфавит Σ = {a, b, c, d}; контрольная подцепочка — dca. Каждая буква c должна иметь непосредственно слева d и непосредственно справа a. Остальные буквы используются свободно. Ограничение проверяется для всех вхождений c.

В ЛР2 автомат Мура завершал обработку по служебному символу z и выдавал y1, y2 либо y3. В этой работе грамматики и распознаватели работают над Σ, поэтому z не вводится: окончание слова определяется исчерпанием входа.

Распознаваемый язык L = {w ∈ Σ* | каждое вхождение c в w имеет контекст dca}. Слова без c и пустое слово принадлежат L: нарушения в них нет. Поэтому y1 и y2 соответствуют принятию, y3 — отказу. Само различие y1 и y2 сохранено в таблице тестов. Для языка только с результатом y2 достаточно убрать правила S → λ и A → λ и оставить конечными только C и D.

ЛР2	Условие	Результат в ЛР3
y1	Буква c отсутствует	Accept
y2	Есть c, все вхождения правильные	Accept
y3	Есть нарушение контекста c	Reject

Титульный лист сохранён из исходного документа без изменений по просьбе автора. Название и номер текущей работы приведены на этой странице.

2 Конечный автомат и ручная грамматика

Из исходного автомата оставлены рабочие состояния q0–q4. Состояние ошибки q5 заменено отсутствием перехода; конечные выходные состояния q6–q8 и переходы по z не нужны. Соответствие: q0 ↔ S, q1 ↔ A, q2 ↔ B, q3 ↔ C, q4 ↔ D.

Состояние	Смысл	Конечное
S	с ещё не было; последний символ не d	Да
A	с ещё не было; последний символ d	Да
B	Прочитано dc; ожидается a	Нет
C	Контекст dca завершён; последний символ не d	Да
D	Контекст dca завершён; последний символ d	Да

Грамматика G = (N, Σ, P, S), где N = {S, A, B, C, D}. Переход X —x→ Y задаёт продукцию X → xY; конечное состояние X задаёт X → λ. Это праволинейная грамматика типа 3, являющаяся частным случаем КС-грамматики типа 2.

S → aS | bS | dA | λ

A → aS | bS | dA | cB | λ

B → aC

C → aC | bC | dD | λ

D → aC | bC | dD | cB | λ

Всего 19 продукций: 15 переходных и 4 завершающих. Для B нет правила B → λ: слово, оканчивающееся на dc, недопустимо. Нет переходов по c из S и C: перед c должен стоять d.

2.1 Обоснование языка

Правила по a, b и d сохраняют сведения о последней букве и о наличии завершённого контекста. Буква c выводится только из A и D, то есть после d, и переводит вывод в B. Единственное правило B → aC принудительно добавляет a. Следовательно, каждый выведенный c имеет нужных соседей. Обратно, для любого слова без нарушений последовательно применимы правила, соответствующие его символам; после полного чтения достигается S, A, C или D и применяется λ-правило. Значит, L(G) = L.

3 Граф и таблица конечного распознавателя

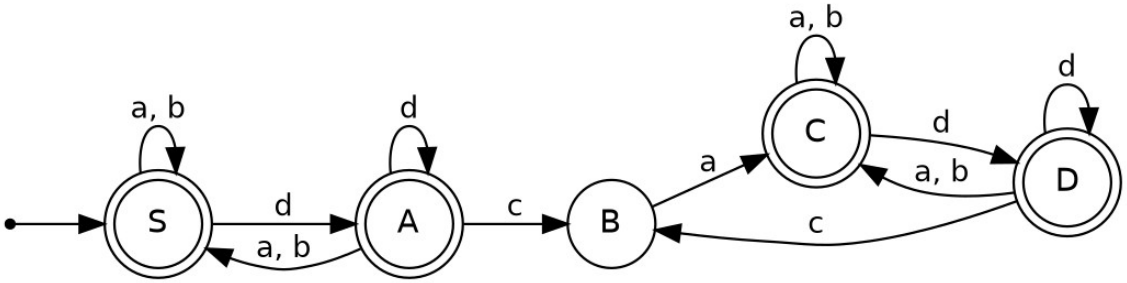


Рисунок 1 — Граф конечного распознавателя; двойной круг — конечное состояние

Состояние	a	b	c	d
S	S	S	—	A
A	S	S	B	A
B	C	—	—	—
C	C	C	—	D
D	C	C	B	D

Начальное состояние — S; F = {S, A, C, D}. Прочерк означает отсутствие перехода и отказ. Файл 02_исходный_KA.jff содержит эту модель.

3.1 Преобразование грамматики в автомат

При стандартном преобразовании праволинейной грамматики сохраняются переходы X —x→ Y для X → xY. Вводится единое конечное состояние F; для X → λ добавляется переход X —λ→ F. Получается 6 состояний, 15 буквенных и 4 λ-перехода. Единственное конечное состояние — F. Это преобразование программно выполнено в модели 03_KA_из_грамматики.jff.

Переходы в F: S —λ→ F, A —λ→ F, C —λ→ F, D —λ→ F. Из B и из F переходов в F нет. Ранний переход в F при непрочитанном входе не принимает слово: вход должен быть исчерпан.

3.2 Грамматика, восстановленная из полученного автомата

Каждый буквенный переход снова даёт X → xY, каждый λ-переход в F даёт X → F; добавляется F → λ. В файле 04_грамматика_из_KA.jff используются правила:

S → aS | bS | dA | F

A → aS | bS | dA | cB | F

B → aC

C → aC | bC | dD | F

D → aC | bC | dD | cB | F

F → λ

После подстановки F → λ получаются исходные правила G. Поэтому исходный КА, ручная грамматика и восстановленная грамматика эквивалентны.

4 Выводы слов и деревья разбора

Для dca: S ⇒ dA ⇒ dcB ⇒ dcaC ⇒ dca. Для dcadca: S ⇒ dA ⇒ dcB ⇒ dcaC ⇒ dcadD ⇒ dcadcB ⇒ dcadcaC ⇒ dcadca. Для пустого слова: S ⇒ λ.

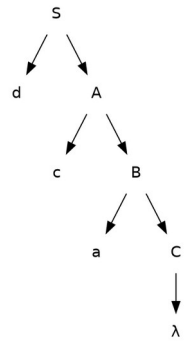


Рисунок 2 — Дерево вывода слова dca

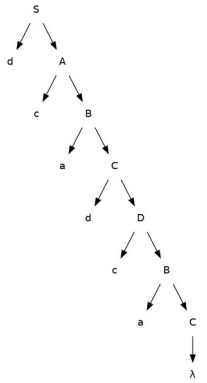


Рисунок 3 — Дерево вывода слова dcadca

Листья деревьев слева направо дают проверяемое слово; λ не является входным символом. Деревья построены непосредственно по правилам G.

Пример отказа dc: S ⇒ dA ⇒ dcB. Для завершения требовалось бы B ⇒* λ, но единственное правило B → aC добавляет букву a. Поэтому слово dc не выводится. Для aca вывод блокируется после S ⇒ aS: правила S → cX нет.

5 Построение МП-автомата по КС-грамматике

Используется стандартный нисходящий недетерминированный МПА. Q = {start, init, loop, accept}, Σ = {a,b,c,d}, Γ = {a,b,c,d,S,A,B,C,D,$}. Начальное состояние start; конечное — accept. В начале стек пуст, $ обозначает его дно. Вершина стека записывается слева.

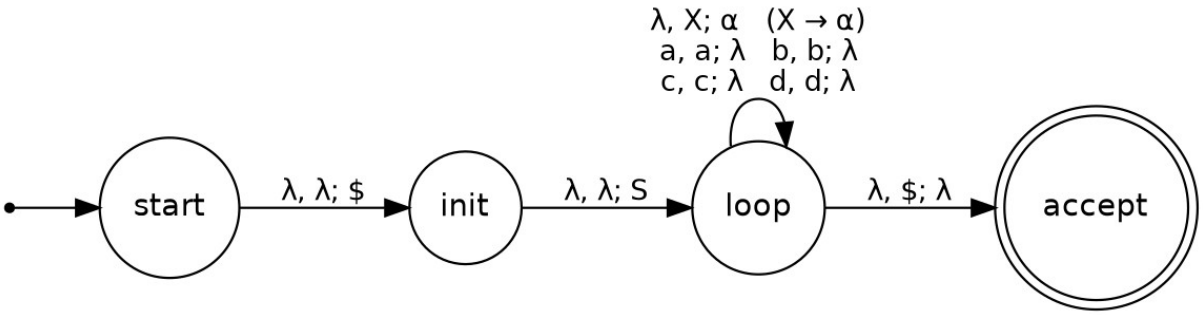


Рисунок 4 — Схема МП-автомата; метка: вход, снять; положить

Откуда → куда	Вход	Снять	Положить	Назначение
start → init	λ	λ	$	Создать дно стека
init → loop	λ	λ	S	Положить начальный нетерминал
loop → loop	λ	X	α	Для каждого X → α
loop → loop	x	x	λ	Для каждого x ∈ {a,b,c,d}
loop → accept	λ	$	λ	Завершить разбор

Каждой из 19 продукций соответствует отдельная петля (λ, X; α). Например, S → dA даёт (λ, S; dA), B → aC — (λ, B; aC), C → λ — (λ, C; λ). Ещё 4 петли сопоставляют терминалы входа и стека. Всего 4 состояния и 26 переходов.

При замене X на xY символ x становится вершиной стека и считывается следующим. Нетерминал Y остаётся под ним. Выбор продукции недетерминирован: слово принимается, если хотя бы одна ветвь достигает accept после полного чтения входа. Наличие неудачных ветвей не означает отказ для всего слова.

В корректной ветви стек имеет вид X$, xX$ либо $. Удаление последнего нетерминала по λ-правилу и затем $ допустимо для принятия только при пустом остатке входа. Состояние accept не имеет исходящих переходов.

6 Анализ содержимого стека

6.1 Успешная ветвь для слова dca

Шаг	Состояние	Остаток	Стек	Действие
0	start	dca	λ	Начало
1	init	dca	$	Положить $
2	loop	dca	S$	Положить S
3	loop	dca	dA$	S → dA
4	loop	ca	A$	Считать d
5	loop	ca	cB$	A → cB
6	loop	a	B$	Считать c
7	loop	a	aC$	B → aC
8	loop	λ	C$	Считать a
9	loop	λ	$	C → λ
10	accept	λ	λ	Удалить $; принять

6.2 Отказ для dc

После начальных действий и чтения dc получается конфигурация (loop, λ, B$). Терминал c прочитан, но его правый сосед ещё не подтверждён. Единственная замена B → aC создаёт стек aC$. Вход пуст, поэтому снять a сопоставлением невозможно; конечное состояние недостижимо. Нельзя удалить B по λ: такого правила нет.

6.3 Пустое слово и слово без c

Пустое слово: (start, λ, λ) ⊢ (init, λ, $) ⊢ (loop, λ, S$) ⊢ (loop, λ, $) ⊢ (accept, λ, λ). Это соответствует результату y1 прошлой работы. Для abdda успешный вывод имеет вид S ⇒ aS ⇒ abS ⇒ abdA ⇒ abddA ⇒ abddaS ⇒ abdda; стек после каждого сопоставления содержит текущий нетерминал над $.

6.4 Ошибка после правильного контекста

При чтении dcac после dca достигается (loop, c, C$). Правило C → λ может завершить ветвь с непрочитанным c, но такая ветвь не является принимающей. Остальные правила C начинаются с a, b или d и не совпадают с c. Значит, правильный первый фрагмент не скрывает последующую ошибку.

7 Проверка эквивалентности

Проверены исходный конечный распознаватель, грамматика G, грамматика из преобразованного КА и МПА. В таблице КА — исходный конечный автомат; G′ — восстановленная грамматика; «+» означает принятие, «−» — отказ. Входы совпадают с тестами ЛР2 после удаления служебного z.

№	Слово	ЛР2	КА	G	G′	МПА
1	λ	y1	+	+	+	+
2	abdda	y1	+	+	+	+
3	d	y1	+	+	+	+
4	dca	y2	+	+	+	+
5	bdcaab	y2	+	+	+	+
6	dcadca	y2	+	+	+	+
7	ddcaa	y2	+	+	+	+
8	dcadcab	y2	+	+	+	+
9	c	y3	−	−	−	−
10	ca	y3	−	−	−	−
11	dc	y3	−	−	−	−
12	dcb	y3	−	−	−	−
13	dcd	y3	−	−	−	−
14	aca	y3	−	−	−	−
15	dccaa	y3	−	−	−	−
16	dcac	y3	−	−	−	−
17	dcaacda	y3	−	−	−	−
18	abcdca	y3	−	−	−	−

Дополнительно независимый программный перебор проверил все 5461 слов длиной от 0 до 6 над Σ: результат каждого из четырёх распознавателей совпал с прямой проверкой соседей всех букв c. Для грамматик использован поиск левосторонних выводов, для МПА — поиск конфигураций (состояние, позиция входа, стек). Проверка МПА моделирует недетерминизм, а не только заранее выбранную успешную ветвь.

Перебор проверяет реализации на указанном конечном наборе. Эквивалентность для слов любой длины следует из преобразований правил и доказательства языка в разделе 2.1.

Проверки выполнены программно по сохранённым моделям. Интерфейс JFLAP в этой среде не запускался; рисунки являются графами и деревьями моделей, а не снимками рабочих окон. Фактические результаты GUI и снимки окон в отчёте не заявляются.

8 Воспроизведение проверки в JFLAP

1. Открыть 01_грамматика.jff через File → Open. Убедиться, что первой стоит продукция с S. В Input → Brute Force Parser ввести dca, нажать Start и просмотреть вывод. Повторить для dcadca, dc и пустого поля ввода. Зафиксировать дерево принятого слова и результат отказа.

2. Для открытой грамматики выбрать Convert → Convert Right-Linear Grammar to FA → Show All → Export. Сравнить результат с моделью 03: буквенные переходы совпадают с таблицей, λ-переходы соединяют S, A, C, D с общим финалом. Имена и расположение состояний могут отличаться.

3. Открыть 02_исходный_КА.jff и полученный КА. В Input → Multiple Run ввести слова из таблицы. Пустое слово задаётся пустым полем, символ λ буквально не вводится. Проверить совпадение принятия и отказа.

4. Для обратного преобразования полученного КА использовать Convert → Convert to Grammar. Сравнить правила с 04_грамматика_из_КА.jff с учётом переименования нетерминалов. Проверить те же слова через Brute Force Parser.

5. Открыть 05_МП_автомат.jff. Запустить Multiple Run на тех же входах, затем пошаговый разбор dca и dc. Сравнить стек с разделом 6. Для принятия требуется полный разбор входа и достижение accept. Сохранить снимки общего графа, результатов и конфигураций стека, если они требуются преподавателем.

Выводы

Для варианта 13 построена праволинейная КС-грамматика, распознающая слова, в которых каждая c находится между d и a. На основе автомата ЛР2 получен конечный распознаватель; выполнены программные преобразования грамматики в КА и КА обратно в грамматику. По правилам грамматики построен недетерминированный МП-автомат.

Все четыре модели дали одинаковые результаты для 18 контрольных слов и при полном переборе 5461 слов длиной до 6. Разбор стека показывает, почему после dc обязательно требуется a и почему преждевременное завершение ветви не принимает непрочитанный остаток входа. Подготовлены файлы для воспроизведения проверки в JFLAP.

Использованные материалы

1. Григорьева Т. Н., Исаков. Учебно-методические материалы из предоставленного PDF: с. 29, 33–37, 48; лабораторная работа 3 — с. 36–37.
2. Отчёт «теор форм яз2.docx»: лабораторная работа № 2, вариант 13, автомат Мура для dca.